FILE: C:\HCL_TRAINING\client\src\pages\PipelinePage.jsx

LINES: 337 | SIZE: 12.5 KB | LANGUAGE: JSX

MODULE: 7-Stage Drag-and-Drop Kanban Recruitment Board with Undo Support

```
1  | import React, { useState, useEffect } from 'react';
2  | import { useSearchParams, Link } from 'react-router-dom';
3  | import {
4  |   KanbanSquare,
5  |   Users,
6  |   Search,
7  |   Filter,
8  |   ArrowRight,
9  |   RotateCcw,
10 |   CheckCircle2,
11 |   XCircle,
12 |   MoreVertical,
13 |   Plus,
14 |   Briefcase,
15 |   User,
16 |   Clock,
17 |   Sparkles,
18 | } from 'lucide-react';
19 | import { api } from '../api/client';
20 | import { ScoreBadge } from '../components/common/ScoreBadge';
21 |
22 | export const PipelinePage = () => {
23 |   const [searchParams] = useSearchParams();
24 |   const initialJobId = searchParams.get('jobId') || 'all';
25 |
26 |   const [jobs, setJobs] = useState([]);
27 |   const [selectedJob, setSelectedJob] = useState(initialJobId);
28 |   const [pipelineData, setPipelineData] = useState(null);
29 |   const [search, setSearch] = useState('');
30 |   const [loading, setLoading] = useState(true);
31 |
32 |   // Bulk Selection
33 |   const [selectedApps, setSelectedApps] = useState([]);
34 |   const [showBulkModal, setShowBulkModal] = useState(false);
35 |   const [bulkTargetStage, setBulkTargetStage] = useState('Shortlisted');
36 |
37 |   // Last Moved Application for Undo
38 |   const [lastMovedAppId, setLastMovedAppId] = useState(null);
39 |
40 |   const STAGES = ['New', 'Screening', 'Shortlisted', 'Interview', 'Offer', 'Hired', 'Rej
   >  ected'];
41 |
42 |   useEffect(() => {
43 |     fetchJobs();
44 |   }, []);
45 |
46 |   useEffect(() => {
47 |     fetchPipeline();
48 |   }, [selectedJob, search]);
49 |
50 |   const fetchJobs = async () => {
51 |     try {
52 |       const res = await api.get('/jobs');
53 |       if (res.success) setJobs(res.data || []);
54 |     } catch (e) {}
55 |   };
56 |
57 |   const fetchPipeline = async () => {
58 |     setLoading(true);
59 |     try {
60 |       const query = new URLSearchParams();
61 |       if (selectedJob !== 'all') query.append('jobId', selectedJob);
62 |       if (search) query.append('search', search);
63 |
64 |       const res = await api.get(`/pipeline?${query.toString()}`);
65 |       if (res.success) {
```

```
66 |          setPipelineData(res.data);
67 |        }
68 |      } catch (err) {
69 |        console.error(err);
70 |      } finally {
71 |        setLoading(false);
72 |      }
73 |    };
74 |
75 |    const handleMoveStage = async (appId, targetStage) => {
76 |      try {
77 |        const res = await api.patch(`/pipeline/${appId}/stage`, { stage: targetStage });
78 |        if (res.success) {
79 |          setLastMovedAppId(appId);
80 |          fetchPipeline();
81 |        }
82 |      } catch (err) {
83 |        alert(err.message || 'Failed to move candidate stage');
84 |      }
85 |    };
86 |
87 |    const handleUndo = async () => {
88 |      if (!lastMovedAppId) return;
89 |      try {
90 |        const res = await api.post(`/pipeline/${lastMovedAppId}/undo`, {});
91 |        if (res.success) {
92 |          setLastMovedAppId(null);
93 |          fetchPipeline();
94 |        }
95 |      } catch (err) {
96 |        alert(err.message || 'Undo failed');
97 |      }
98 |    };
99 |
100 |    const handleBulkAction = async () => {
101 |      if (selectedApps.length === 0) return;
102 |      try {
103 |        const res = await api.post('/pipeline/bulk-stage', {
104 |          applicationIds: selectedApps,
105 |          targetStage: bulkTargetStage,
106 |        });
107 |        if (res.success) {
108 |          setSelectedApps([]);
109 |          setShowBulkModal(false);
110 |          fetchPipeline();
111 |        }
112 |      } catch (err) {
113 |        alert(err.message || 'Bulk update failed');
114 |      }
115 |    };
116 |
117 |    const toggleSelectApp = (appId) => {
118 |      setSelectedApps((prev) =>
119 |        prev.includes(appId) ? prev.filter((id) => id !== appId) : [...prev, appId]
120 |      );
121 |    };
122 |
123 |    const columns = pipelineData?.columns || {};
124 |
125 |    return (
126 |      <div className="space-y-6 max-w-full mx-auto">
127 |        {/* Header */}
128 |        <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4 pb
    >   -2 border-b border-border">
129 |          <div>
130 |            <span className="text-xs font-semibold uppercase tracking-wider text-forest-70
    >   0">
131 |              Candidate Pipeline Management
132 |            </span>
133 |            <h1 className="font-editorial text-3xl font-bold text-ink mt-0.5">
134 |              Recruitment Kanban Board
135 |            </h1>
```

```
136 |           <p className="text-xs text-ink-muted mt-1">
137 |             Track candidates across stages, reassign recruiter ownership, and execute bu
    >   lk actions.
138 |           </p>
139 |         </div>
140 |
141 |         <div className="flex items-center gap-2.5">
142 |           {lastMovedAppId && (
143 |             <button
144 |               onClick={handleUndo}
145 |               className="inline-flex items-center gap-1.5 px-3 py-1.5 bg-amber-50 border
    >    border-amber-200 text-amber-900 rounded-lg text-xs font-semibold hover:bg-amber-100 tra
    >   nsition-colors"
146 |             >
147 |               <RotateCcw className="w-3.5 h-3.5" />
148 |               <span>Undo Last Move</span>
149 |             </button>
150 |           )}
151 |
152 |           {selectedApps.length > 0 && (
153 |             <button
154 |               onClick={() => setShowBulkModal(true)}
155 |               className="inline-flex items-center gap-1.5 px-3.5 py-1.5 bg-forest-500 te
    >   xt-white rounded-lg text-xs font-medium shadow-sm hover:bg-forest-600 transition-colors"
156 |             >
157 |               <span>Bulk Action ({selectedApps.length})</span>
158 |             </button>
159 |           )}
160 |         </div>
161 |       </div>
162 |
163 |       {/* Filter and Search Bar */}
164 |       <div className="surface-card rounded-2xl p-4 flex flex-col md:flex-row md:items-ce
    >   nter justify-between gap-3">
165 |         <div className="flex items-center gap-3">
166 |           <label className="text-xs font-semibold text-ink shrink-0">Filter by Job Openi
    >   ng:</label>
167 |           <select
168 |             value={selectedJob}
169 |             onChange={(e) => setSelectedJob(e.target.value)}
170 |             className="bg-[#FBFBF9] border border-border rounded-lg text-xs font-medium
    >   text-ink px-3 py-1.5 focus:outline-none focus:border-forest-500 min-w-[240px]"
171 |           >
172 |             <option value="all">All Jobs ({jobs.length})</option>
173 |             {jobs.map((j) => (
174 |               <option key={j._id} value={j._id}>
175 |                 {j.title}
176 |               </option>
177 |             ))}
178 |           </select>
179 |         </div>
180 |
181 |         <div className="relative w-full sm:w-64">
182 |           <Search className="w-4 h-4 text-ink-subtle absolute left-3 top-1/2 -translate-
    >   y-1/2" />
183 |           <input
184 |             type="text"
185 |             placeholder="Search candidate name or email..."
186 |             value={search}
187 |             onChange={(e) => setSearch(e.target.value)}
188 |             className="w-full bg-[#FBFBF9] border border-border rounded-lg pl-9 pr-3 py-
    >   1.5 text-xs text-ink focus:outline-none focus:border-forest-500"
189 |           />
190 |         </div>
191 |       </div>
192 |
193 |       {/* 7-Stage Horizontal Scrollable Kanban Columns */}
194 |       <div className="flex gap-4 overflow-x-auto pb-6 pt-1 min-h-[550px]">
195 |         {STAGES.map((stageName) => {
196 |           const col = columns[stageName] || { count: 0, candidates: [] };
197 |
198 |           return (
```

```
199 |             <div
200 |               key={stageName}
201 |               className="w-72 shrink-0 flex flex-col rounded-2xl bg-[#F8F9F6] border bor
    >  der-border overflow-hidden"
202 |             >
203 |               {/* Column Header */}
204 |               <div className="p-3.5 border-b border-border flex items-center justify-bet
    >  ween bg-white">
205 |                 <div className="flex items-center gap-2">
206 |                   <span className="font-semibold text-xs text-ink">{stageName}</span>
207 |                   <span className="text-[10px] font-mono px-2 py-0.5 rounded-full bg-sag
    >  e-100 text-forest-700 font-bold border border-sage-200">
208 |                     {col.count}
209 |                   </span>
210 |                 </div>
211 |               </div>
212 |
213 |               {/* Candidates Column Body */}
214 |               <div className="p-2.5 flex-1 space-y-2.5 overflow-y-auto max-h-[600px]">
215 |                 {col.candidates.length === 0 ? (
216 |                   <div className="text-center py-10 text-[11px] text-ink-subtle italic">
217 |                     No candidates in {stageName}
218 |                   </div>
219 |                 ) : (
220 |                   col.candidates.map((card) => {
221 |                     const isChecked = selectedApps.includes(card._id);
222 |
223 |                     return (
224 |                       <div
225 |                         key={card._id}
226 |                         className={`surface-card rounded-xl p-3 border transition-all ho
    >  ver:border-forest-300 hover:shadow-card group relative ${
227 |                           isChecked ? 'ring-2 ring-forest-500 bg-forest-50/20' : ''
228 |                         }`}
229 |                       >
230 |                         <div className="flex items-start justify-between gap-2 mb-1.5">
231 |                           <div className="flex items-center gap-1.5 min-w-0">
232 |                             <input
233 |                               type="checkbox"
234 |                               checked={isChecked}
235 |                               onChange={() => toggleSelectApp(card._id)}
236 |                               className="rounded text-forest-600 focus:ring-forest-500 c
    >  ursor-pointer shrink-0"
237 |                             />
238 |                             <Link
239 |                               to={`/candidates/${card.candidate?._id}`}
240 |                               className="font-bold text-xs text-ink hover:text-forest-70
    >  0 truncate"
241 |                             >
242 |                               {card.candidate?.name}
243 |                             </Link>
244 |                           </div>
245 |                           <ScoreBadge score={card.score} size="sm" />
246 |                         </div>
247 |
248 |                         <span className="text-[10px] text-ink-muted block truncate mb-2"
    >  >
249 |                           {card.job?.title} &bull; {card.candidate?.totalExperienceYears
    >  }y exp
250 |                         </span>
251 |
252 |                         {/* Top 2 skills */}
253 |                         <div className="flex flex-wrap gap-1 mb-2.5">
254 |                           {(card.candidate?.skills || []).slice(0, 2).map((s) => (
255 |                             <span
256 |                               key={s.name}
257 |                               className="text-[9px] px-1.5 py-0.2 rounded bg-sage-50 tex
    >  t-ink border border-sage-200"
258 |                             >
259 |                               {s.name}
260 |                             </span>
261 |                           ))}
```

```
                        </div>

                        {/* Stage Progression Selector */}
                        <div className="pt-2 border-t border-border flex items-center ju
  > stify-between text-[11px]">
                          <span className="text-[10px] text-ink-subtle">Move to:</span>
                          <select
                            value={card.stage}
                            onChange={(e) => handleMoveStage(card._id, e.target.value)}
                            className="bg-[#FBFBF9] border border-border rounded text-[1
  > 0px] px-1.5 py-0.5 text-ink font-medium focus:outline-none focus:border-forest-500 curso
  > r-pointer"
                          >
                            {STAGES.map((s) => (
                              <option key={s} value={s}>
                                {s}
                              </option>
                            ))}
                          </select>
                        </div>
                      </div>
                    );
                  })}
                </div>
              </div>
            );
          })}
        </div>

        {/* Bulk Action Confirmation Modal */}
        {showBulkModal && (
          <div className="fixed inset-0 z-50 bg-black/40 backdrop-blur-sm flex items-cente
  > r justify-center p-4">
            <div className="bg-white rounded-2xl shadow-dropdown border border-border max-
  > w-sm w-full p-6 space-y-4">
              <h3 className="font-editorial text-lg font-bold text-ink">
                Bulk Move Candidates
              </h3>
              <p className="text-xs text-ink-muted">
                Move {selectedApps.length} selected candidate(s) simultaneously to a targe
  > t stage.
              </p>

              <div>
                <label className="block text-xs font-semibold text-ink mb-1.5">Target Stag
  > e</label>
                <select
                  value={bulkTargetStage}
                  onChange={(e) => setBulkTargetStage(e.target.value)}
                  className="w-full bg-[#FBFBF9] border border-border rounded-lg px-3 py-2
  >  text-xs text-ink font-medium"
                >
                  {STAGES.map((s) => (
                    <option key={s} value={s}>
                      {s}
                    </option>
                  ))}
                </select>
              </div>

              <div className="pt-2 flex justify-end gap-2">
                <button
                  type="button"
                  onClick={() => setShowBulkModal(false)}
                  className="px-3.5 py-1.5 border border-border rounded-lg text-xs font-me
  > dium text-ink hover:bg-sage-50"
                >
                  Cancel
                </button>
                <button
                  type="button"
```

```
325 |                 onClick={handleBulkAction}
326 |                 className="px-4 py-1.5 bg-forest-500 hover:bg-forest-600 text-white roun
    >  ded-lg text-xs font-medium shadow-sm"
327 |               >
328 |                 Confirm Move
329 |               </button>
330 |             </div>
331 |           </div>
332 |         </div>
333 |       )}
334 |     </div>
335 |   );
336 | };
337 |
```